DESIGN AND ANALYSIS OF ALGORITHMS

Coding Challenge Task – Q7

MINIMUM SPANNING TREE – NETWORK DESIGN

Comparison and implementation of Prim’s and Kruskal’s Algorithms

Course / Activity	CO4 – Algorithmic Coding Challenge
Question	Q7 – Minimum Spanning Tree – Network Design
Application	ISP city-to-city network / minimum cable cost
Algorithms	Prim’s Algorithm and Kruskal’s Algorithm
Recommended Structure	Adjacency list + min-heap for Prim; edge list + DSU for Kruskal

This report explains the MST problem, compares Prim’s and Kruskal’s algorithms, provides a feasible Python solution, performs a sample dry run, analyzes complexity, and interprets the result for practical network design.

1. Problem Statement

An Internet Service Provider (ISP) needs to connect multiple cities using communication cables. Each possible connection has a known installation cost. The objective is to connect every city while using the minimum possible total cable cost and avoiding unnecessary cycles. This can be modeled as a weighted, connected, undirected graph, where cities are vertices and cable links are weighted edges.

Goal: Construct a Minimum Spanning Tree (MST) and determine its minimum total cost.

2. Objectives

• Model the ISP network as a weighted undirected graph.

• Understand how Prim's and Kruskal's algorithms construct an MST.

• Compare their algorithmic approach, data structures, and complexity.

• Implement an efficient solution suitable for competitive programming.

• Interpret the MST cost as the minimum cable installation cost.

3. Minimum Spanning Tree Concept

A Minimum Spanning Tree is a subset of edges that connects all vertices of a connected weighted graph, contains exactly V−1 edges, contains no cycle, and has minimum possible total edge weight.

• Every city is reachable from every other city through the selected links.

• The selected network has exactly V − 1 cable connections for V cities.

• No redundant cycle is introduced.

• The sum of selected cable costs is minimum among all spanning trees.

4. Prim's Algorithm

Prim's algorithm starts from any vertex and grows a single tree. At each step it selects the cheapest edge connecting a vertex already in the tree to a vertex outside the tree. A min-priority queue is used to efficiently identify the cheapest candidate edge.

Data Structures

• Adjacency list: stores neighboring cities and cable costs.

• Min-heap / priority queue: retrieves the cheapest candidate edge.

• Visited array: prevents a city from being added more than once.

Prim's Pseudocode

```
PrimMST(G):
choose a start vertex
mark start as visited
insert its incident edges into min_heap
while min_heap is not empty and MST has V−1 edges:
remove minimum-cost edge (w, u, v)
if v is not visited:
```

```
add (u, v, w) to MST
add w to total cost
mark v visited
insert v's outgoing edges into min_heap
return MST and total cost
```

5. Kruskal’s Algorithm

Kruskal’s algorithm considers edges globally from lowest cost to highest cost. It adds an edge if that edge does not create a cycle. A Disjoint Set Union (DSU), also called Union-Find, efficiently detects whether two vertices already belong to the same component.

Data Structures

• Edge list: stores every cable as (cost, city1, city2).

• Sorting: orders all cable connections by increasing cost.

• DSU / Union-Find: supports Find and Union operations for cycle detection.

Kruskal’s Pseudocode

```
KruskalMST(G):
sort all edges by increasing weight
initialize DSU for all vertices
for each edge (u, v, w):
if Find(u) != Find(v):
add edge to MST
cost += w
Union(u, v)
stop after V−1 edges
return MST and total cost
```

6. Prim vs Kruskal Comparison

Feature	Prim’s Algorithm	Kruskal’s Algorithm
Strategy	Grows one tree from a starting city	Builds a forest by globally choosing cheapest edges
Main structure	Adjacency list + min-heap	Edge list + sorting + DSU
Cycle handling	Visited set/array	DSU / Union-Find
Typical complexity	O(E log V) with binary heap	O(E log E), effectively O(E log V) for simple connected graphs
Best practical use	Often convenient for dense graphs / adjacency representation	Often convenient for sparse graphs / edge-list representation
Starting vertex	Required	Not required
Disconnected graph	Produces a spanning tree only if connected; otherwise a partial result	Produces a minimum spanning forest

For an ISP network represented naturally as a list of available cable links, Kruskal is a strong choice because the input is already an edge list and DSU makes cycle detection efficient. Prim is equally valid and is particularly convenient when the graph is represented using adjacency lists.

7. Feasible Python Solution – Kruskal’s Algorithm

```
class DSU:
def __init__(self, n):
self.parent = list(range(n))
self.rank = [0] * n

def find(self, x):
if self.parent[x] != x:
self.parent[x] = self.find(self.parent[x])
return self.parent[x]

def union(self, a, b):
a, b = self.find(a), self.find(b)
if a == b:
return False

if self.rank[a] < self.rank[b]:
a, b = b, a

self.parent[b] = a
if self.rank[a] == self.rank[b]:
self.rank[a] += 1
return True


def kruskal(n, edges):
edges.sort(key=lambda x: x[2])
dsu = DSU(n)
mst = []
cost = 0

for u, v, w in edges:
if dsu.union(u, v):
mst.append((u, v, w))
```

```
cost += w

if len(mst) == n - 1:
break

if len(mst) != n - 1:
return None, None

return mst, cost


n = int(input("Enter number of cities: "))
m = int(input("Enter number of connections: "))

edges = []
print("Enter each connection as: city1 city2 cost")

for _ in range(m):
u, v, w = map(int, input().split())
edges.append((u, v, w))

mst, cost = kruskal(n, edges)

if mst is None:
print("MST cannot be formed: graph is disconnected.")
else:
print("Selected connections:")
for u, v, w in mst:
print(u, "-", v, ":", w)
print("Minimum Network Cost =", cost)
```

8. Sample Input and Output

Sample network with 5 cities and 7 possible cable connections:

```
Enter number of cities: 5
Enter number of connections: 7
Enter each connection as: city1 city2 cost
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9
```

Output

```
Selected connections:
0 - 1 : 2
1 - 2 : 3
1 - 4 : 5
0 - 3 : 6
Minimum Network Cost = 16
```

9. Dry Run

Order	Edge	Cost	Decision
1	0–1	2	Select
2	1–2	3	Select
3	1–4	5	Select
4	0–3	6	Select
5	1–3	8	Reject – creates cycle
6	2–4	7	Reject – creates cycle
7	3–4	9	Not required; MST already has 4 edges

The selected edges connect all five cities with exactly four edges and no cycle. The total cost is 2 + 3 + 5 + 6 = 16.

10. Complexity Analysis

Algorithm	Time Complexity	Space Complexity
Prim + binary min-heap	O(E log V)	O(V + E)
Kruskal + sorting + DSU	O(E log E)	O(V + E)

Kruskal is dominated by sorting the edges. With DSU path compression and union by rank, cycle checks are nearly constant amortized time. Prim with a binary heap performs well when the graph is represented through adjacency lists.

11. Test Case Handling

Test	Network condition	Expected result
1	Connected graph	MST exists; cost is minimum
2	Single city, no edges	Cost = 0
3	Two cities with one edge	That edge is selected
4	Connected graph with cycles	Cheapest cycle-free edges selected
5	Disconnected graph	MST cannot be formed
6	Equal-cost edges	Any valid minimum-cost MST is acceptable

12. Practical Network Design Interpretation

• The MST minimizes initial cable installation cost while maintaining connectivity.

• Each selected edge represents a cable link that contributes directly to the network budget.

• Unselected edges are not necessarily useless; they may be retained as backup/redundant links in a real ISP network.

• Real deployments must also consider latency, bandwidth, reliability, geographical constraints, maintenance, and redundancy. Therefore, an MST is an excellent cost baseline

but not always the final production topology.

• Kruskal is a practical choice when an ISP receives a large list of candidate links with installation costs. Prim is useful when network adjacency information is naturally available.

13. Rubric-Oriented Evaluation

Criterion	Evidence in solution
Problem Understanding – 20%	Models cities and cable costs as a weighted graph; defines MST, constraints, practical interp
Algorithm – 25%	Compares Prim and Kruskal and selects an efficient greedy MST strategy.
Code Implementation – 20%	Uses clean Python, DSU with path compression and union by rank, input validation, and disc
Competitive Performance – 20%	Kruskal runs in O(E log E) and DSU makes cycle detection efficient.
Test Case Handling – 15%	Considers connected, disconnected, single-city, two-city, cyclic, and equal-weight cases.

14. GitHub Repository Structure

```
CO4_AT1/
├── Question_1/
│ ├── Source_Code/
│ │ └── mst_kruskal.py
│ └── Report/
│ └── Q7_MST_Network_Design_Report.pdf
└── README.md
```

Include screenshots of the source code, terminal execution, sample output, and additional test cases in the Report folder. Submit only the GitHub repository link through Google Classroom.

15. Conclusion

Both Prim’s and Kruskal’s algorithms are correct greedy methods for constructing a Minimum Spanning Tree. For an ISP network represented as candidate cable links, Kruskal’s algorithm with sorting and DSU provides a simple and efficient implementation. For the sample network, the MST connects all five cities using four links with a minimum cable cost of 16. In practical network design, the MST gives the minimum-cost connectivity baseline; additional redundancy and service-quality constraints can then be incorporated for a production network.